

Documento de Arquitectura de Software (SAD) — Menu Bites

Versión: 2.0.0 | **Alcance:** Arquitectura técnica, patrones de diseño e infraestructura del sistema de gestión de restaurantes Menu Bites.

1. VISIÓN GENERAL DEL SISTEMA

Menu Bites es una plataforma SaaS (Software as a Service) multitenant diseñada para digitalizar la operación de restaurantes. El sistema abarca desde la gestión administrativa global hasta la interacción directa del cliente mediante menús digitales con acceso vía QR.

Principios de Diseño

- Multitenancy por discriminador de columna:** Todos los recursos de negocio están vinculados a un `restaurant_id`, aislando los datos de cada local a nivel de base de datos.
- Single source of truth:** `schema.prisma` es la fuente de verdad para todos los tipos y relaciones.
- Realtime first:** Las vistas operativas (KDS, Waiter, Local Dashboard) usan suscripciones Supabase Realtime en lugar de polling.
- Edge security:** La validación de sesión y roles ocurre en el middleware de Next.js antes de que la solicitud llegue a la lógica de negocio.

2. ARQUITECTURA DE SOFTWARE

2.1 Estructura de Monorepo

El proyecto utiliza una arquitectura de monorepo gestionada con **Turborepo** y **Workspaces de npm**, permitiendo compartir lógica, componentes y configuración entre las diferentes aplicaciones.

Aplicaciones (`apps/`):

App	Rol	Usuario Destino
<code>admin-dashboard</code>	Panel de administración global de la plataforma SaaS	SUPER_ADMIN
<code>local-dashboard</code>	Panel operativo por restaurante: menú, pedidos, reportes, branding	ADMIN
<code>kitchen-kds</code>	Pantalla de cocina para gestión de tickets en tiempo real	COCINA
<code>waiter-terminal</code>	Terminal móvil para toma de pedidos y gestión de mesas	GARZON
<code>cashier-dashboard</code>	Panel de caja para cierre de cuentas y cobro	CAJERO
<code>customer-portal</code>	Portal web del cliente final, accedido vía código QR de mesa	CLIENTE

App	Rol	Usuario Destino
mobile	Aplicación móvil (en desarrollo)	GARZON / CLIENTE

Paquetes Compartidos (**packages/**):

Paquete	Descripción
@menu-bites/ui	Biblioteca de componentes React compartidos (Button , Modal , Table , Badge , OrderTicket , Card , cn , etc.)
@menu-bites/auth	Cliente Supabase instanciado y helpers de sesión (getSession , signOut , supabase)
@menu-bites/store	Estado global compartido (Zustand o similar) para datos cross-app

Diagrama de Estructura del Monorepo

```
%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#6366f1',
    'primaryTextColor': '#ffffff',
    'primaryBorderColor': '#4f46e5',
    'lineColor': '#94a3b8',
    'secondaryColor': '#10b981',
    'tertiaryColor': '#f8fafc',
    'fontFamily': 'Inter, sans-serif',
    'fontSize': '14px'
  }
}}%%
graph TD
  classDef appNode fill:#6366f1,stroke:#4f46e5,color:#fff;
  classDef pkgNode fill:#10b981,stroke:#059669,color:#fff;
  classDef rootNode fill:#1e293b,stroke:#0f172a,color:#fff,stroke-width:2px;

  subgraph Monorepo ["Monorepo (Turborepo + npm Workspaces)"]
    direction TB
    A[Apps]:::rootNode --> B[admin-dashboard]:::appNode
    A --> C[local-dashboard]:::appNode
    A --> D[kitchen-kds]:::appNode
    A --> E[waiter-terminal]:::appNode
    A --> F[cashier-dashboard]:::appNode
    A --> G[customer-portal]:::appNode
    A --> H[mobile]:::appNode

    P[Packages]:::rootNode --> UI["@menu-bites/ui"]:::pkgNode
    P --> AUTH["@menu-bites/auth"]:::pkgNode
  end
```

```

P --> STORE["@menu-bites/store"]:::pkgNode

B -->|usa| UI
C -->|usa| UI
D -->|usa| UI
E -->|usa| UI
F -->|usa| UI
G -->|usa| UI

B -->|usa| AUTH
C -->|usa| AUTH
D -->|usa| AUTH
E -->|usa| AUTH
F -->|usa| AUTH
G -->|usa| AUTH
end

```

3. MODELO DE DATOS (ERD — Resumen)

La persistencia se basa en un esquema relacional en PostgreSQL, gestionado con Prisma ORM. La tabla **restaurants** actúa como eje central (Tenant) de todas las operaciones. Para el ERD completo con campos detallados, ver [DATABASE_SCHEMA.md](#).

```

%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#4f46e5',
    'primaryTextColor': '#ffffff',
    'primaryBorderColor': '#4338ca',
    'lineColor': '#818cf8',
    'secondaryColor': '#10b981',
    'tertiaryColor': '#f1f5f9',
    'fontFamily': 'Inter, sans-serif'
  }
}}%%
erDiagram
    RESTAURANT ||--o{ USER : "gestiona"
    RESTAURANT ||--o{ CATEGORY : "contiene"
    RESTAURANT ||--o{ MENU_ITEM : "ofrece"
    RESTAURANT ||--o{ TABLE : "posee"
    RESTAURANT ||--o{ ORDER : "registra"
    RESTAURANT ||--o{ INVENTORY : "monitorea"
    RESTAURANT ||--o{ THEME : "aplica"
    PLAN ||--o{ RESTAURANT : "asigna"

    CATEGORY ||--o{ MENU_ITEM : "clasifica"
    MENU_ITEM ||--o{ ORDER_ITEM : "incluido en"
    ORDER ||--o{ ORDER_ITEM : "contiene"
    TABLE ||--o{ ORDER : "asociada a"
    INVENTORY ||--o{ MENU_ITEM_INGREDIENT : "es insumo de"

```

```
MENU_ITEM ||--o{ MENU_ITEM_INGREDIENT : "requiere"
MENU_ITEM ||--o{ MENU_ITEM_EXTRA : "posee"
ORDER_ITEM ||--o{ ORDER_ITEM_EXTRA : "incluye"
```

4. STACK TECNOLÓGICO

Frontend y Framework

Tecnología	Versión	Uso
Next.js	14+	App Router, SSR/CSR híbrido, middleware de Edge, API Routes
React	18+	Motor de UI
TypeScript	5+	Seguridad de tipos en todo el monorepo
Tailwind CSS	3+	Framework de estilos, glassmorphism, responsividad

Backend y Servicios

Tecnología	Uso
Supabase PostgreSQL	Base de datos relacional con RLS habilitado
Supabase Auth	Gestión de identidad, JWT, sesiones, app_metadata (role, restaurant_id)
Supabase Realtime	Suscripciones a cambios en tablas (orders, tables) para actualizaciones instantáneas
Supabase Storage	Almacenamiento de imágenes de menú y logotipos (bucket menu-images)
Prisma ORM	Definición de tipos, migraciones y validación de esquema

Infraestructura y Tooling

Herramienta	Uso
Turborepo	Orquestación de build del monorepo con cache distribuido
Vercel	Hosting y CI/CD de todas las apps
npm Workspaces	Gestión de dependencias inter-paquetes

5. FLUJO DE DATOS Y ESTADOS

5.1 Ciclo de Vida de un Pedido

```
%%{init: {
  'theme': 'base',
  'themeVariables': {
```

```

    'primaryColor': '#f8fafc',
    'primaryTextColor': '#1e293b',
    'primaryBorderColor': '#e2e8f0',
    'lineColor': '#6366f1',
    'actorBkg': '#6366f1',
    'actorBorder': '#4f46e5',
    'actorTextColor': 'ffffff',
    'noteBkgColor': '#f1f5f9',
    'noteTextColor': '#1e293b',
    'fontFamily': 'Inter, sans-serif'
  }
}%%
sequenceDiagram
    autonumber
    participant C as Cliente/Garzon
    participant API as Next.js API
    participant DB as Supabase DB
    participant RT as Realtime Hub
    participant KDS as Kitchen KDS

    C->>API: POST /api/local/orders (Crear Pedido)
    API->>DB: Validar sesion y guardar orden
    DB-->>RT: Disparar evento INSERT en orders
    RT-->>KDS: Notificacion instantanea
    KDS->>KDS: Actualizar lista de tickets
    KDS->>DB: PUT orders/{id} (status: PREPARING)
    DB-->>RT: Evento UPDATE en orders
    RT-->>C: Notificar cambio de estado

```

5.2 Máquina de Estados del Pedido

```

%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#6366f1',
    'primaryTextColor': 'ffffff',
    'primaryBorderColor': '#4f46e5',
    'lineColor': '#64748b',
    'secondaryColor': '#10b981',
    'fontFamily': 'Inter, sans-serif'
  }
}%%
stateDiagram-v2
    [*] --> PENDING : Cliente o Garzon crea pedido
    PENDING --> VALIDATED : Garzon valida disponibilidad
    PENDING --> REJECTED : Garzon rechaza pedido
    VALIDATED --> PREPARING : Cocina inicia preparacion (KDS)
    PREPARING --> READY : Cocina marca como listo (KDS)
    READY --> DELIVERED : Garzon entrega al cliente
    DELIVERED --> [*]
    REJECTED --> [*]

```

5.3 Gestión de Seguridad y Aislamiento (Multi-tenancy)

```
%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#1e293b',
    'primaryTextColor': '#ffffff',
    'primaryBorderColor': '#0f172a',
    'lineColor': '#64748b',
    'secondaryColor': '#ef4444',
    'tertiaryColor': '#10b981',
    'fontFamily': 'Inter, sans-serif',
    'fontSize': '14px'
  }
}}%%
flowchart TD
    classDef secureNode fill:#10b981,stroke:#059669,color:#fff;
    classDef gateNode fill:#f59e0b,stroke:#d97706,color:#fff;
    classDef alertNode fill:#ef4444,stroke:#dc2626,color:#fff;
    classDef startNode fill:#1e293b,stroke:#0f172a,color:#fff;

    Req[Solicitud HTTP]:::startNode --> MW[Next.js Middleware
Edge]:::gateNode
    MW -->|Extraer Slug| Tenant{Tenant valido?}:::gateNode
    Tenant -->|No| Err[404 Not Found]:::alertNode
    Tenant -->|Si| Session{Sesion activa?}:::gateNode
    Session -->|No| Login[Redirigir a Login]:::secureNode
    Session -->|Si| Role{RBAC: Rol permitido?}:::gateNode
    Role -->|Insuficiente| Deny[403 Acceso Denegado]:::alertNode
    Role -->|Autorizado| App[Renderizar Aplicacion]:::secureNode
    App --> DB[(Supabase DB)]:::startNode
    DB -->|RLS Policy activa| Final[Datos aislados por
restaurant_id]:::secureNode
```

5.4 Motor de Marca Dinámica (Branding Engine)

```
%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#ffffff',
    'primaryTextColor': '#1e293b',
    'primaryBorderColor': '#e2e8f0',
    'lineColor': '#6366f1',
    'secondaryColor': '#6366f1',
    'tertiaryColor': '#f8fafc',
    'fontFamily': 'Inter, sans-serif'
  }
}}%%
graph LR
```

```

classDef dbNode fill:#1e293b,stroke:#0f172a,color:#fff;
classDef uiNode fill:#6366f1,stroke:#4f46e5,color:#fff;
classDef processNode fill:#f8fafc,stroke:#e2e8f0,color:#1e293b;

subgraph "Configuracion (Admin / Local Dashboard)"
  S[Guardar Tema en Branding Lab]:::uiNode
end

subgraph "Propagacion a Aplicaciones Cliente"
  L[Cargar config de Supabase]:::processNode --> P[Procesar colores a HSL]:::processNode
  P --> I[Inyectar CSS Custom Properties]:::processNode
  I --> U[UI actualizada en tiempo real]:::uiNode
end

S -->|Escritura en restaurant_themes| DB[(Supabase DB)]:::dbNode
DB -->|Lectura al iniciar app| L

```

6. INFRAESTRUCTURA Y DESPLIEGUE

6.1 Diagrama de Despliegue

```

%%{init: {
  'theme': 'base',
  'themeVariables': {
    'primaryColor': '#1e293b',
    'primaryTextColor': '#ffffff',
    'primaryBorderColor': '#0f172a',
    'lineColor': '#6366f1',
    'secondaryColor': '#10b981',
    'tertiaryColor': '#f8fafc',
    'fontFamily': 'Inter, sans-serif'
  }
}}%%
flowchart LR
  classDef gitNode fill:#24292e,stroke:#1a1e22,color:#fff;
  classDef ciNode fill:#f59e0b,stroke:#d97706,color:#fff;
  classDef appNode fill:#6366f1,stroke:#4f46e5,color:#fff;
  classDef infraNode fill:#10b981,stroke:#059669,color:#fff;

  GH[GitHub Repository]:::gitNode -->|Push a branch| CI[Vercel CI/CD]:::ciNode
  CI -->|Build Turborepo| BUILD[Build Cache]:::ciNode

  BUILD --> APP1[admin-dashboard.vercel.app]:::appNode
  BUILD --> APP2[local-dashboard.vercel.app]:::appNode
  BUILD --> APP3[kitchen-kds.vercel.app]:::appNode
  BUILD --> APP4[waiter-terminal.vercel.app]:::appNode
  BUILD --> APP5[cashier-dashboard.vercel.app]:::appNode
  BUILD --> APP6[customer-portal.vercel.app]:::appNode

```

```

APP1 & APP2 & APP3 & APP4 & APP5 & APP6 -->|HTTPS + Supabase JS SDK|
SUPA[(Supabase Cloud)]:::infraNode

SUPA --> PG[PostgreSQL + RLS]:::infraNode
SUPA --> AUTH[Auth Service]:::infraNode
SUPA --> RT[Realtime Websockets]:::infraNode
SUPA --> ST[Storage Buckets]:::infraNode

```

6.2 Estrategia de CI/CD

- **Vercel:** Plataforma de hosting para todas las aplicaciones del monorepo. El despliegue automático se activa con cada push a las ramas configuradas (**main** → producción, **develop** → staging).
- **Turborepo Build Cache:** Turborepo detecta qué paquetes cambiaron y solo recompila los afectados, reduciendo tiempos de build hasta en un 80%.
- **Variables de Entorno:** Cada aplicación tiene sus propias variables de entorno en Vercel (**NEXT_PUBLIC_SUPABASE_URL**, **NEXT_PUBLIC_SUPABASE_ANON_KEY**). El **SUPABASE_SERVICE_ROLE_KEY** nunca se expone al cliente.

6.3 Estrategia de Dominios

El sistema utiliza subrutas o subdominios para diferenciar organizaciones, orquestado mediante el middleware de Next.js:

app.menubites.com/{slug}/dashboard	→ Local Dashboard del restaurante
app.menubites.com/{slug}/kds	→ Kitchen KDS
app.menubites.com/{slug}/waiter	→ Waiter Terminal
app.menubites.com/{slug}/cashier	→ Cashier Dashboard
menu.menubites.com/{slug}	→ Customer Portal (público)